Neural_Networks

Task	Convolutional	Transformer
Dense Prediction (segmentation & depth estimation)	FCN, U-Net, monoDepth	DPT, SETR
Image Recognition	ResNet-50, EfficientNet	ViT, DinoV2
Object Detection	YOLO, Faster R-CNN, CenterNet, FCOS	DETR
Image Captioning	CNN+LSTM,	CLIP
Keypoint detection (local feature matching)	Superpoint, superglue	Loftr

	Low dimensional states	High-dimensional states
Static dataset	Classic supervised learning (ML)	Deep supervised learning (DL)
Agent/environment interaction	Tabular reinforcement learning	Deep reinforcement learning

Activation functions

1. Code: mytorch/activations.py

```python
import numpy as np

class Activation:
    def __init__(self):
        self.x = None
        self.update_weights = False

    def forward(self, z):
        raise NotImplementedError()

    def backward(self, dLdx):
        raise NotImplementedError()

class Identity(Activation):
    def forward(self, z):
        self.x = z
        return self.x

    def backward(self, dLdx):
        dxdz = np.ones_like(self.x)
        dLdz = dLdx * dxdz
        return dLdz

class ReLU(Activation):
    def forward(self, z):
        self.x = np.maximum(z, np.zeros_like(h))
        return self.x

    def backward(self, dLdx):
        dxdz = np.zeros_like(self.x)
        dxdz[np.where(self.x > 0)] = 1
        dLdz = dLdx * dxdz
        return dLdz

class Tanh():
    """
    Modified for RNN Classifier.
    """
    def forward(self, z):
        # self.x = (np.exp(z) - np.exp(-z)) / (np.exp(z) + np.exp(-z))
        self.x = np.tanh(z)  # more stable to overflow
        return self.x

    def backward(self, dLdx, h_t=None):
        if h_t is None:
            dxdz = 1 - self.x ** 2
        else:
            dxdz = 1 - h_t ** 2
        dLdz = dLdx * dxdz
        return dLdz

class Sigmoid(Activation):
    def forward(self, z):
        self.x = 1 / (1 + np.exp(-z))
```

```
        return self.x

    def backward(self, dLdx):
        dLdz = dLdx * self.x * (1 - self.x)
        return dLdz

class Softmax(Activation):
    def forward(self, z):
        exp_z = np.exp(z - np.max(z, axis = 1, keepdims=True))
        sum_exp_z = np.sum(exp_z, axis = 1, keepdims=True)
        self.x = exp_z / sum_exp_z
        return self.x

    def backward(self, dLdx):
        N = self.x.shape[0]
        C = self.x.shape[1]
        dLdz = np.zeros(shape=(N, C))
        for k in range(N):
            J = np.zeros(shape=(C, C))
            for i in range(C):
                for j in range(C):
                    if i == j:
                        J[i, j] = self.x[k, i] * (1 - self.x[k, i])
                    else:
                        J[i, j] = - self.x[k, i] * self.x[k, j]
            dLdz[k, :] = dLdx[k, :] @ J
        return dLdz

if __name__ == "__main__":
    z = np.random.randint(-5,5,(3,4))
    relu = ReLU()
    x = relu.forward(z)
    dLdx = np.random.randint(0,5,size=x.shape)
    dLdz = relu.backward(dLdx)
    print("z = ", z)
    print("x = ", x)
    print("dLdx = ", dLdx)
    print("dLdz = ", dLdz)
    print("")
    softmax = Softmax()
    x = softmax.forward(z)
    dLdz = softmax.backward(dLdx)
    print("z = ", z)
    print("x = ", x)
    print("dLdx = ", dLdx)
    print("dLdz = ", dLdz)
```

Computational Graph

1. When we draw computation graph **circles represents function**, they don't represent variables. The variables are arrows between the functions, the circles are functions. So each of the circles are mathematical operations, it has inputs and outputs, and its job is to compute the derivative of its outputs wrt. to its inputs.
2. Graph structure backpropagation, also called automatic differentiation:
    1. do the following at each layer $f(x_f) = y_f$, starting with the last function, where $\delta = 1$.
    2. $\frac{\partial L}{\partial \theta_f} = \frac{\partial f}{\partial \theta_f}\delta_{y_f}$, local derivative $\frac{\partial f}{\partial \theta_f}$, upstream derivative δ_{y_f}.

Flattening

1. a batch of images is a 4D tensor (N, C, H, W).
2. An image $I \in \mathrm{R}^{H\times W\times 3}$ or feature map $f \in \mathrm{R}^{H\times W\times C}$ can be flattened into $I \in \mathrm{R}^{HW\times 3}$, $f \in \mathrm{R}^{HW\times C}$.
3. Flattening dimensions is often used when we are doing **matrix multiplications** such as when preparing input to MLP, $f_1 \in \mathrm{R}^{HW\times C_1} \rightarrow MLP \rightarrow f_2 \in \mathrm{R}^{HW\times C_2}$.
4. During flattening the dimensions to be flattened are 'aggregated' into one dimension, for example (3, 4, 5) into (3, 20).
5. Code:

```
# torch.flatten

import torch

# torch.flatten
x = torch.rand(3, 4, 5)
a = x.flatten(0)
b = x.flatten(1)
print("x.shape", x.shape)
print("a.shape", a.shape)
print("b.shape", b.shape)
```

Out:

```
x.shape torch.Size([3, 4, 5])
a.shape torch.Size([60])
b.shape torch.Size([3, 20])
```

6. Example:
    1. mfccs shape: (T, 28)
    2. during training using MLP:
        1. x: frames: (N, T, D)
        2. N: batch_size
        3. T: time_window: $2 * \text{context\_window} + 1$
        4. D: 28 (dimension)
        5. reshape x, (N, $T * D$)->mlp->phonemes (n, 42)

Incorporating information in neural nets

1. Most of the times in neural nets we need to incorporate information using either summing or concatenation.
2. Based on information theory, as long as the information is there it should be usable. From information standpoint it does not really matter either we use summing or concatenation.
   2. Sum: less memory requirement
   3. Concatenate: more memory

Labels

1. multi-class classification labels: use softmax (already inside nn.CrossEntropyLoss) and cross-entropy loss with no spatial dimensions: $C := \text{num\_classes}$. $\underbrace{(N,)}_{\substack{\text{values ranging from} \\ \text{0 to C-1}}} \rightarrow \underbrace{(N, C)}_{\substack{\text{will be converted to one-hot} \\ \text{inside nn.CrossEntropyLoss}}} \rightarrow \begin{bmatrix} [0,0,1,0,0,0,0,0,0] \\ [0,0,0,0,0,1,0,0,0] \\ [1,0,0,0,0,0,0,0,0] \end{bmatrix}$.
2. Segmentation labels (same as classification with two spatial dimensions): $\underbrace{(N, H, W)}_{\substack{\text{values ranging from} \\ \text{0 to C-1}}} \rightarrow \underbrace{(N, C, H, W)}_{\text{one-hot in nn.CrossEntropyLoss}}$. $\text{pred:}(N, C, H, W)$.
3. multi-label classification: use sigmoid (nn.Sigmoid) and binary cross-entropy loss (nn.BCELoss) or use linear and nn.BCEWithLogitsLoss: $(N, C) \rightarrow \begin{bmatrix} [1,0,0,1,0,0,0,0,0] \\ [0,0,1,1,0,0,0,0,0] \\ [0,0,0,0,0,0,1,0,0] \end{bmatrix}$. Note: multi-label classification is just binary classification per channel.
4. Note: difference between sigmoid+bceloss vs softmax+crossentropy [6]: The difference may lie in the background class. Sigmoid + BCE does not need an extra class for the background. The background class is just an all-zero one-hot vector. Softmax + CE needs an extra class for the background. So the total class in DETR is num_classes + 1 and the total class in Deformable DETR is num_classes. The performance between the two methods may be similar.

Losses

1. Losses are always if not most of the time consist of taking the difference of
    1. two tensors (pred and target)
    2. filtering (like causal masks, indicator function like in YOLO loss)
    3. some kind of multiplication (element-wise, matmul)
    4. some kind of reduction (summation or max or min across one or more than one dimensions) to get norm (for example Frobenius norm)
2. MLE: either maximize the log likelihood $\sum p \log(p)$ or minimize the negative log likelihood as loss function: $\sum -p \log(p)$. Note: Why is p called the likelihood? because often the parameter to be

Mean Squared Error (MSE) Loss

1. MSE loss:
    1. vector form: $L_i = \frac{1}{m} \sum_i (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$, $\mathbf{x}_i \in \mathrm{R}^d$.
    2. matrix form: $L = \sum_i \sum_j (\hat{x}_{ij} - x_{ij}) \odot (\hat{x}_{ij} - x_{ij})$, $x : (N, D)$. Notice that the square is translated into element-wise multiplication $\odot$ in the matrix form.

KL divergence

1. KL divergence: take the expectation of the logarithm of the ratio of p to q as if you were to assume that x is sampled through p.
2. $KL(P||Q) = \sum_{i=1}^{n} p_i \log\left(\frac{p_i}{q_i}\right)$. p is the distribution we want to learn (target distribution), q is our model prediction distribution.

Cross entropy

1. In practice, delta:=$\frac{\partial L}{\partial z}$ is computed inside the loss function e.g. CrossEntropyLoss class in classification tasks in pytorch. Also notice that delta is proportional to the error term $y_{pred} - y_{target}$. Because the Softmax function is already defined inside the CrossEntropyLoss, during model initialization the last layer of the model for classification is often a linear layer.
2. Cross entropy loss: $p \log \frac{p}{q} = p \log p - p \log q = 0 - p \log q$. Where p is the probability of target, q is the predicted probability $\hat{p}$. If p is one-hot p then will be 1 for the true label 0 for everywhere else, we're left with $L_{ce} = -\log \hat{p}$ for those predictions with true labels. $\frac{\partial L}{\partial \hat{p}} = -\frac{1}{\hat{p}}$. This means during backpropagation, increasing $\hat{p}$ will lead to a decrease in the loss since probability $\hat{p}$ is strictly positive i.e. $\hat{p} > 0$. The increase in the probability of the correct class will eventually lead to decrease in all other class (incorrect classes), since the probability computed by softmax can only sum up to 1. Note: In pytoch, default Cross Entropy Loss accepts logits as inputs.

NLL

1. Negative Log-Likelihood error is equivalent to MSE if you have unit variance $\sigma^2 = I$.
2. If you learn the variance, then it ends up being weighted MSE.
3. Alternatively one can use LogSoftmax layer and apply NLLoss instead of CrossEntropyLoss.
4. NLL: $\min_w \sum_{i=1}^{D} -\log P(y^{(i)}|x^{(i)}, w) \ldots \text{min. log likelihood.}$

Code:

```
import numpy as np


class Loss:
    def __init__(self):
        self.x = None
        self.y = None

    def forward(self, x, y):
        raise NotImplementedError()

    def backward(self):
        raise NotImplementedError()


class MSELoss(Loss):
    def forward(self, x, y):
        # x: shape: (N, C)
        # y: shape: (N, C)
        self.x = x
        self.y = y
        N = x.shape[0]
        C = x.shape[1]
        squared_error = (x - y) * (x - y)
        sum_of_squared_error = np.sum(squared_error, axis=(0, 1))
        mean_squared_error_loss = sum_of_squared_error / (N * C)
        # print("mean_squared_error_loss ", mean_squared_error_loss)
        return mean_squared_error_loss

    def backward(self):
        N = self.x.shape[0]
        C = self.x.shape[1]
        dLdx = 2 * (self.x - self.y) / (N * C)
        # print("dLdx", dLdx)
        return dLdx


class CrossEntropyLoss(Loss):
    def forward(self, x, y):
        # x: shape: (N, C)
        # y: shape: (N, C)
        self.y = y
        exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
        sum_exp_x = np.sum(exp_x, axis=1, keepdims=True)
        self.x = exp_x / sum_exp_x
        N = x.shape[0]
        cross_entropy = np.sum(-y * np.log(self.x), axis=1)
        sum_cross_entropy = np.sum(cross_entropy, axis=0)
        mean_cross_entropy = sum_cross_entropy / N
        return mean_cross_entropy

    def backward(self):
```

```
        N = self.x.shape[0]
        dLdx = (self.x - self.y) / N
        return dLdx


if __name__ == "__main__":
    x = np.random.randint(0, 5, size=(2, 5))
    y = np.random.randint(0, 5, size=(2, 5))
    mseloss = MSELoss()
    L = mseloss.forward(x, y)
    dLdxN = mseloss.backward()

    print("x", x)
    print("y", y)
    print("L ", L)
    print("dLdxN", dLdxN)
    print("")

    x = np.zeros(shape=(2, 5))
    y = np.zeros_like(x)
    y[0, 2] = 1
    y[1, 1] = 1

    crossentropy = CrossEntropyLoss()
    L = crossentropy.forward(x, y)
    dLdxN = crossentropy.backward()

    print("x", x)
    print("y", y)
    print("L ", L)
    print("dLdxN", dLdxN)
```

Discriminative Models

MLP for Regression

1. Code:

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchsummaryX import summary
import gc
from tqdm.auto import tqdm
import matplotlib.pyplot as plt


"""
Non-linear Regression
We will implement an MLP to fit a Sine function
"""
class MyDataset(Dataset):
    def __init__(self, xs, ys):
        self.xs = xs
        self.ys = ys
        self.length = len(self.xs)

    def __len__(self):
        return self.length

    def __getitem__(self, idx):
        return self.xs[idx], self.ys[idx]


class MyFirstModel(nn.Module):
    def __init__(self):

        super().__init__()

        # Create layers. In this case just a standard MLP
        self.layers = nn.Sequential(
            nn.Linear(1, 128),
            # nn.BatchNorm1d(128),
            nn.ReLU(),
            # nn.BatchNorm1d(128),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 1),
            nn.Identity(),
        )

        # self.fc1 = nn.Linear(1, 10)
        # self.bn1 = nn.BatchNorm1d(10)
        # self.relu1 = nn.ReLU()
        # self.fc2 = nn.Linear(10, 24)
        # self.relu2 = nn.ReLU()
        # self.fc3 = nn.Linear(24, 10)
```

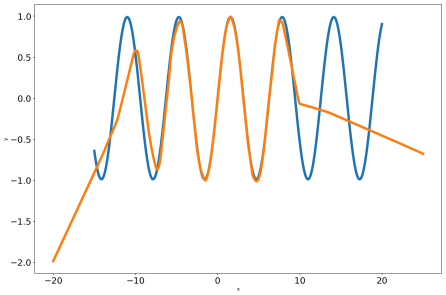
```
        # self.relu3 = nn.ReLU()
        # self.fc4 = nn.Linear(10, 1)
        # self.identity1 = nn.Identity()

    def forward(self, x):
        # x = self.fc1(x)
        # x = self.bn1(x)
        # x = self.relu1(x)
        # x = self.fc2(x)
        # x = self.relu2(x)
        # x = self.fc3(x)
        # x = self.relu3(x)
        # x = self.fc4(x)
        # x = self.identity1(x)
        return self.layers(x)


def normalize(x):
    mu = torch.mean(x)
    std = torch.std(x)
    x -= mu
    x /= std
    return x


if __name__ == "__main__":
    if torch.cuda.is_available():
        device = torch.device("cuda")
    else:
        device = torch.device("cpu")
    print("Device :", device)
    # Prepare dataset
    # Add a dimension to input so that the shape is (N, C), C=1
    x_train = torch.linspace(-15, 20, 300).unsqueeze(1)
    y_train = torch.exp(-0.01 * torch.ones_like(x_train)) * torch.sin(x_train)

    # Normalize inputs
    # x_train = normalize(x_train)
    # y_train = normalize(y_train)

    print(x_train[:20])
    print(y_train[:20])
    # y = torch.exp(minus_one) * torch.sin(x) + torch.randn(len(x))
    print(x_train.shape)
    print(y_train.shape)
    dataset = MyDataset(x_train, y_train)
    dataloader = DataLoader(dataset=dataset, batch_size=32, drop_last=True)
    print("len(dataloader)", len(dataloader))
    for i, data in enumerate(dataloader):
        # print(i, data)
        print(len(data))
        print(data[0].shape)
        print(data[1].shape)
        break

    # Initialize model
    model = MyFirstModel().to(device)

    # Initialize criterion
    criterion = torch.nn.MSELoss()

    # Initialize optimizer
    # optimizer = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
    # optimizer = torch.optim.Adam(model.parameters(), lr=1e-2, weight_decay=1e-5)

    # Print model's state_dict
    print("Model's state_dict:")
    for param_tensor in model.state_dict():
        print(param_tensor, "\t", model.state_dict()[param_tensor].size())

    # Print optimizer's state_dict
    print("Optimizer's state_dict")
    for var_name in optimizer.state_dict():
        print(var_name, "\t", optimizer.state_dict()[var_name])

    summary(model, x_train.to(device))

    torch.cuda.empty_cache()
    gc.collect()
    # Train model
    n_epochs = 500
    for _ in range(n_epochs):
        model.train()
        tloss = 0
        batch_bar = tqdm(
            total=len(dataloader),
            dynamic_ncols=True,
            leave=False,
            position=0,
```

```
            desc="Train",
        )
        for i, (x, y) in enumerate(dataloader):

            # Initialize Gradients
            optimizer.zero_grad()

            # Move data to device (ideally GPU)
            x = x.to(device)
            y = y.to(device)

            # Forward propagation
            y_pred = model(x)

            # Loss calculation
            loss = criterion(y_pred, y)

            # Backward propagation
            loss.backward()

            # Gradient descent
            optimizer.step()

            tloss += loss.item() * x.size(0)

            batch_bar.set_postfix(loss="{:.04f}".format(float(tloss / (i + 1))))
            batch_bar.update()

            del x, y, y_pred
            torch.cuda.empty_cache()

        batch_bar.close()
        # Get loss over one epoch
        tloss /= len(dataloader.dataset)

    x_test = torch.linspace(-20, 25, 500).unsqueeze(1)
    # x_test = normalize(x_test)

    # model.eval()
    with torch.inference_mode():
        y_test = model.forward(x_test.to(device))

    x_train_numpy = x_train.detach().cpu().numpy()
    y_train_numpy = y_train.detach().cpu().numpy()

    x_test_numpy = x_test.detach().cpu().numpy()
    y_test_numpy = y_test.detach().cpu().numpy()

    print(y_test_numpy.shape)

    fig = plt.figure(figsize=(15, 10))
    ax = fig.add_subplot(111)
    ax.plot(x_train_numpy, y_train_numpy, linewidth=5)
    ax.plot(x_test_numpy, y_test_numpy, linewidth=5)
    ax.tick_params(axis="x", labelsize=18)
    ax.tick_params(axis="y", labelsize=18)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    plt.savefig("fig.png")
    # plt.savefig('fig-normalized.png')
    plt.show()

    # Saving model
    torch.save(model, "model.pt")
```

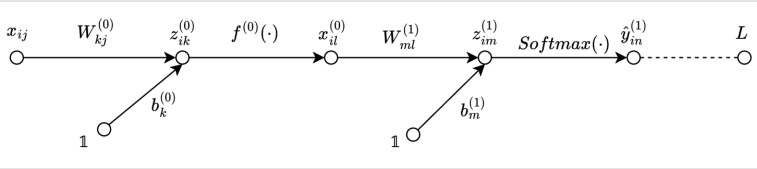
2. Output:

```
Device : cpu
tensor([[-15.0000],
        [-14.8829],
        [-14.7659],
        [-14.6488],
        [-14.5318],
        [-14.4147],
        [-14.2977],
        [-14.1806],
        [-14.0635],
        [-13.9465],
        [-13.8294],
        [-13.7124],
        [-13.5953],
        [-13.4783],
        [-13.3612],
        [-13.2441],
        [-13.1271],
        [-13.0100],
        [-12.8930],
        [-12.7759]])
tensor([[-0.6438],
        [-0.7273],
```

```
        [-0.8007],
        [-0.8633],
        [-0.9140],
        [-0.9522],
        [-0.9773],
        [-0.9891],
        [-0.9874],
        [-0.9721],
        [-0.9435],
        [-0.9021],
        [-0.8482],
        [-0.7828],
        [-0.7066],
        [-0.6208],
        [-0.5265],
        [-0.4250],
        [-0.3176],
        [-0.2059]])
torch.Size([300, 1])
torch.Size([300, 1])
len(dataloader) 9
2
torch.Size([32, 1])
torch.Size([32, 1])
Model's state_dict:
layers.0.weight      torch.Size([128, 1])
layers.0.bias    torch.Size([128])
layers.2.weight      torch.Size([128, 128])
layers.2.bias    torch.Size([128])
layers.4.weight      torch.Size([128, 128])
layers.4.bias    torch.Size([128])
layers.6.weight      torch.Size([1, 128])
layers.6.bias    torch.Size([1])
Optimizer's state_dict
state    {}
param_groups     [{'lr': 0.001, 'betas': (0.9, 0.999), 'eps': 1e-08, 'weight_decay': 1e-05, 'amsgrad': False,
'maximize': False, 'foreach': None, 'capturable': False, 'differentiable': False, 'fused': None,
'decoupled_weight_decay': False, 'params': [0, 1, 2, 3, 4, 5, 6, 7]}]
(500, 1)
```

3. 

MLP for Classification



Influence Diagram

1. A simple MLP with a single hidden layer, i.e. one-hidden-layer neural network, with cross-entropy loss. In this model the output layer is a linear layer which will be explained later.
2. Note: the i index is specially reserved for the sample index, and that the indices j, k, l, m and n for the layer widths are all just dummy variables (no special meaning whatsoever), the 'input' is **not** a layer although historically it was referred to as the 'input layer'.

Forward variables

Symbol	meaning	Shape	
N	batch size	-	
x	input	(N, C^{input})	(batch_size, input_size)
$z^{(0)}$	hidden affine layer	$(N, C^{(0)})$	(batch_size, hidden_size)
$x^{(0)}$	hidden layer	$(N, C^{(0)})$	(batch_size, hidden_size)
$z^{(1)}$	output affine layer	$(N, C^{(1)})$	(batch_size, output_size)

Symbol	meaning	Shape	
$y^{(1)}$	output layer	$(N, C^{(1)})$	(batch_size, output_size)

Model weights

Symbol	meaning	Shape	
$W^{(0)}$	hidden layer weights	$(C^{(0)}, C^{(input)})$	(hidden_size, input_size)
$b^{(0)}$	hidden layer bias	$(C^0, 1)$	(hidden_size, 1)
$W^{(1)}$	output layer weights	$(C^{(1)}, C^{(0)})$	(output_size, hidden_size)
$b^{(1)}$	output layer bias	$(C^{(1)}, 1)$	(output_size, 1)

3. Note: input_size refers to the number of input features. It is different from the number of data samples, which in this case correspond to the batch_size because we are using mini-batch gradient descent for training. For stochastic gradient descent the number of data samples used in each iteration to update the weights would be 1.
4. Scalar form
    1. Forward
        1. $z_{ik}^{(0)} = \sum_j W_{kj}^{(0)} x_{ij} + b_k^{(0)}$
        2. $x_{il}^{(0)} = f^{(0)}(z_{ik}^{(0)})$
        3. $z_{im}^{(1)} = \sum_l W_{ml}^{(1)} x_{il}^{(0)} + b_m^{(1)}$
        4. $y_{in}^{(1)} = Softmax(z_{im}^{(1)}) = \frac{e^{z_{in}^{(1)}}}{\sum_{\mathrm{m}} e^{z_{\mathrm{im}}^{(1)}}}$
        5. $L = \frac{1}{N} \sum_i \sum_n y_{in} \log \frac{y_{in}}{\hat{y}_{in}^{(1)}}$
        6. where z is also called the logits, the output of the model before being normalized into probabilities.
        7. The loss is a distance measure of how far the prediction score is from the ground truth score. To compare distances of two probability distributions, the Kullback-Leibler divergence is used.
    2. Backward
        1. $L = \frac{1}{N} \sum_i \sum_n \left(y_{in} \log y_{in} - y_{in} \log \hat{y}_{in} \right)$
        2. For one-hot encoding of the ground truth probabilities the loss function will be reduced to cross-entropy loss,
        3. $L = \frac{1}{N} \sum_i \sum_n -y_{in} \log \hat{y}_{in}$
        4. $\frac{\partial L}{\partial z_{im}^{(1)}} = \frac{1}{N} \sum_n \frac{\partial L}{\partial \hat{y}_{in}} \frac{\partial \hat{y}_{in}}{\partial z_{im}} = \frac{1}{N} (\hat{y}_{im} - y_{im})$
    3. Remark on derivative of CE Loss to the logits: To obtain $\frac{\partial L}{\partial z_{im}^{(1)}}$, the Jacobian for the Softmax vector activation is given by,
    4. $$\frac{\partial \hat{y}_{in}}{\partial z_{im}^{(1)}} = \frac{\partial}{\partial z_{im}^{(1)}} \left(\frac{e^{z_{in}^{(2)}}}{\sum_{\mathrm{m}} e^{z_{\mathrm{im}}^{(2)}}} \right) = \begin{cases} \hat{y}_{im}(1 - \hat{y}_{im}), & \text{if } n = m \\ -\hat{y}_{im} \hat{y}_{in}, & \text{if } n \neq m \end{cases}$$
    5. and also $\frac{\partial L}{\partial \hat{y}_{in}} = -\frac{y_{in}}{\hat{y}_{in}}$
    6. Combining the above two equations,
    7. $$\begin{aligned} \frac{\partial L}{\partial z_{im}^{(1)}} &= \frac{1}{N} \left(\sum_{n \neq m} -\frac{y_{in}}{\cancel{\hat{y}_{in}}} (-\hat{y}_{im} \cancel{\hat{y}_{in}}) + -\frac{y_{im}}{\cancel{\hat{y}_{im}}} \cancel{\hat{y}_{im}} (1 - \hat{y}_{im}) \right) \\ &= \frac{1}{N} \left(\sum_{n \neq m} y_{in} \hat{y}_{im} - y_{im} + y_{im} \hat{y}_{im} \right) \\ &= \frac{1}{N} \left(\hat{y}_{im} \underbrace{\sum_n y_{in}}_{=1 \Rightarrow \text{one-hot}} - y_{im} \right) \\ &= \frac{1}{N} (\hat{y}_{im} - y_{im}) \end{aligned}$$ $nbsp;nbsp;$
    8. $\frac{\partial L}{\partial x_{il}^{(0)}} = \sum_m \frac{\partial L}{\partial z_{im}^{(1)}} \frac{\partial z_{im}^{(1)}}{\partial x_{il}^{(0)}} = \sum_m \frac{\partial L}{\partial z_{im}^{(1)}} W_{ml}^{(1)}$
    9. $\frac{\partial L}{\partial W_{ml}^{(1)}} = \sum_i \frac{\partial L}{\partial z_{im}^{(1)}} \frac{\partial z_{im}^{(1)}}{\partial W_{ml}^{(1)}} = \sum_i \frac{\partial L}{\partial z_{im}^{(1)}} x_{il}^{(0)}$
    10. $\frac{\partial L}{\partial b_m^{(1)}} = \sum_i \frac{\partial L}{\partial z_{im}^{(1)}} \cdot 1 \, (\text{scalar one})$

11. $\frac{\partial L}{\partial z_{ik}^{(0)}} = \frac{\partial L}{\partial x_{il}^{(0)}} \frac{\partial x_{il}^{(0)}}{\partial z_{ik}^{(0)}}$, for scalar activation e.g. ReLU, $l = k$. $\frac{\partial L}{\partial z_{ik}^{(0)}} = \frac{\partial L}{\partial x_{ik}^{(0)}} f'(z_{ik}^{(0)})$
12. $\frac{\partial L}{\partial x_{ij}} = \sum_k \frac{\partial L}{\partial z_{ik}^{(0)}} \frac{\partial z_{ik}^{(0)}}{\partial x_{ij}} = \sum_k \frac{\partial L}{\partial z_{ik}^{(0)}} W_{kj}^{(0)}$
13. $\frac{\partial L}{\partial W_{kj}^{(0)}} = \sum_i \frac{\partial L}{\partial z_{ik}^{(0)}} \frac{\partial z_{ik}^{(0)}}{\partial W_{kj}^{(0)}} = \sum_i \frac{\partial L}{\partial z_{ik}^{(0)}} x_{ij}$
14. $\frac{\partial L}{\partial b_k^{(0)}} \sum_i \frac{\partial L}{\partial z_{ik}^{(0)}} \frac{\partial z_{ik}^{(0)}}{\partial b_k^{(0)}} = \sum_i \frac{\partial L}{\partial z_{ik}^{(0)}} \cdot 1 \text{ (scalar one)}$

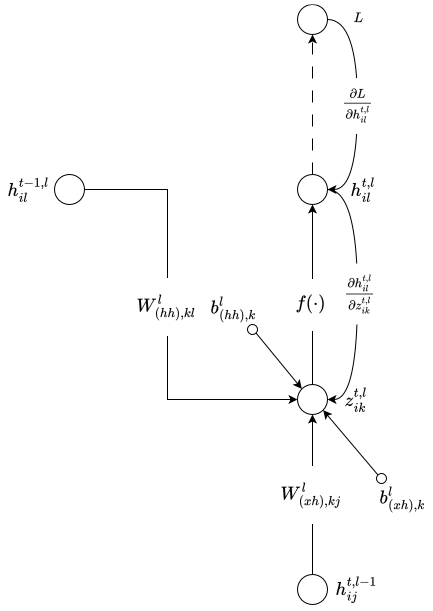
5. Matrix form
    1. Forward
        1. $z^{(0)} = x \cdot W^{(0)T} + b^{(0)}$
        2. $x^{(0)} = f^{(0)}(z^{(0)})$
        3. $z^{(1)} = x^{(0)} \cdot W^{(1)T} + b^{(1)}$
        4. $y_j^{(1)} = Softmax(z_j^{(1)}) = \frac{e^{z_j^{(1)}}}{\sum_j e^{z_j^{(1)}}}$
        5. $L = \frac{\iota_N^T(-y \odot \log \hat{y}^{(1)}) \cdot \iota_C}{N}$
    2. Backward
        1. $\frac{\partial L}{\partial z^{(1)}} = \frac{\hat{y}^{(1)} - y}{N}$
        2. $\frac{\partial L}{\partial x^{(0)}} = \frac{\partial L}{\partial z^{(1)}} \cdot W^{(1)}$
        3. $\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial z^{(1)}}^T x^{(0)}$
        4. $\frac{\partial L}{\partial b^{(1)}} = \frac{\partial L}{\partial z^{(1)}}^T \iota_N$
        5. $\frac{\partial L}{\partial z^{(0)}} = \frac{\partial L}{\partial x^{(0)}} \odot f'^{(0)}(z^{(0)})$
        6. $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z^{(0)}} \cdot W^{(0)}$
        7. $\frac{\partial L}{\partial W^{(0)}} = \frac{\partial L}{\partial z^{(0)}}^T x$
        8. $\frac{\partial L}{\partial b^{(0)}} = \frac{\partial L}{\partial z^{(0)}}^T \cdot \iota_N$
    3. The matrix form is derived from the scalar form. The LHS and RHS of the scalar equations should conform to each other. Summation sign or lack thereof will determine the type of matrix multiplication: normal matrix multiplication $\cdot$, element-wise multiplication $\odot$ or if any broadcasting is needed.

Auto regressive Models

RNN

1. RNNs are only good for sequences with lengths < 30. Vanishing gradients during training (memory loss) becomes a huge problem in RNNs with sequences larger than 30.
2. Task to solve with RNNs:
    1. classification
    2. multi-class classification
    3. generation
3. Advantage of RNN
    1. Suitable for sequential inputs.
    2. The information of order in the input is preserved.
    3. Since RNN is a recursive model, the hidden layer is updated through each time step. Thus, for the phoneme classification task no contexts either in the front and back of the input is required.
4. Issues with RNN:
    1. In RNN, we have a forgetful network, because the gradients may blow up or die out depending on the eigenvalues of the weight matrices (if the magnitude of the eigenvalue is strictly larger than 1), i.e. RNN is not BIBO stable from its stability analysis of its recurrent structure. Note: This happens to the recurrent weights W_{hh} and b_{hh} of the RNN.
    2. Tanh activations in the hidden layers may "hold" the memory up to a few dozen time steps, however the gradient of the weights will still vanish eventually. Note: Using ReLUs will blow it up (Using ReLUs can either blow up or die out the gradients).
    3. RNN will be stable only when the eigenvalues of the **recurrent** weights is 1. However, this is a very strict requirement since the gradients that will hold are these eigenvectors in the subspace with eigenvalue equals 1, all other eigenvectors with eigenvalues not equal to 1 will still explode or die out. Note: see power iteration method.
    4. Short-term memory: Long distance dependencies are not captured by the network.
        1. RNNs are forgeful of their inputs (poor memory).

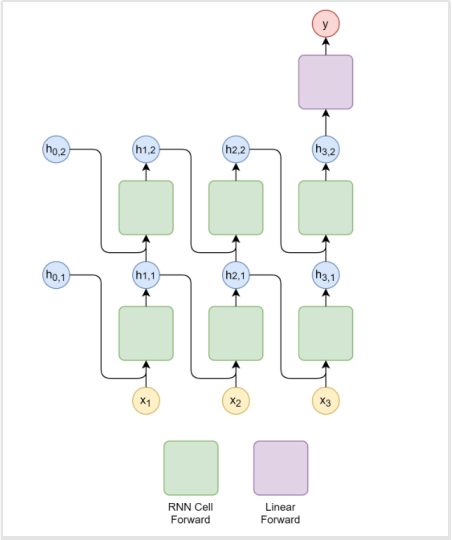
    2. The actual input x is sadly not "remembered" by the RNN, i.e. the hidden layer outputs in the recurrent layer of an RNN will end up at the same values regardless of the input x, after only around a couple of time steps e.g. around 10 time steps. The hidden layer outputs depends on the network parameters W and b mostly and the activation function.
    3. In delayed seq2seq model, the input information is solely encapsulated in the last hidden layer of the decoder.
    4. The memory of an RNN depends on:
        1. The activation function used in the current layers.
        2. The weights of the recurrent layers.
        3. The biases of the recurrent layers.
5. Slow computations
    1. Computationally intensive: RNN should go through each hidden state to reach the output, making it difficult to apply to large sequences of data.
6. Difficulty with parallelization: The sequential nature of RNNs can make it difficult to take advantage of parallel processing to speed up training and inference.

5. Solution:
    1. Remove the weights and replace these with gates. From here we obtain the LSTM.
    2. However LSTM is somewhat hard to train because it has a lot of parameters. For this reason we have the GRU cell which is simpler than the LSTM, however, GRU cell has a weakness that is it cannot count i.e. 1,2,3,...,10, since its output is bounded by a $tanh$.
6. Summary:
    1. Vanishing gradient problem: The gradient signals used to update the weights during training can become very small, making it difficult to train RNNs effectively.
    2. Exploding gradient problem: On the other hand, gradients become too large and cause numeric instability, making it difficult to train RNNs effectively.
7. RNN Cell (CMU 11-785)
    1. 

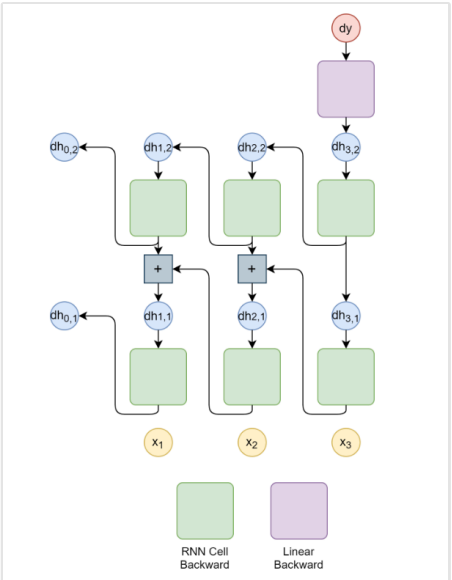
    2. An RNN Cell forward
        1. takes as input: x or h_prev_l, h_prev_t
        2. produces: h_next and cache: (h_next, h_prev_l, h_prev_t, dh_next) (for backprop, in most implementations)
    3. An RNN Cell backward
        1. takes as input, upstream gradient dh_next, h_next, h_prev_l, h_prev_t
        2. produces: dx or dh_prev_l, dh_prev_t
    4. RNN is a parametric function in which itself is conditioned on its own internal state of the previous time step, and the overall effect of this RNN transforms input sequences into output sequences. In colloqual terms, RNNs have a memory of past events which is encoded in this internal state.
    5. Implicit attention: The RNN will attend to some parts of this internal state more than others, i.e. selective memory, selectively looking at something and not others that it received in the past then using those to make decisions. This is why attention and memory are intimately bounded together.
    6. Code: mytorch/rnn_cell.py

```python
import numpy as np
from .activation import *

class RNNCell:

    def __init__(self, input_size, hidden_size):

        self.input_size = input_size
        self.hidden_size = hidden_size

        # Activation function for
        self.activation = Tanh()

        # hidden dimension and input dimension
        h = self.hidden_size
        d = self.input_size

        # Weights and biases
        self.W_ih = np.random.randn(h, d)
        self.W_hh = np.random.randn(h, h)
        self.b_ih = np.random.randn(h)
        self.b_hh = np.random.randn(h)

        self.dW_ih = np.zeros((h, d))
        self.dW_hh = np.zeros((h, h))
        self.db_ih = np.zeros(h)
        self.db_hh = np.zeros(h)

    def init_weights(self, W_ih, W_hh, b_ih, b_hh):
        self.W_ih = W_ih
        self.W_hh = W_hh
        self.b_ih = b_ih
        self.b_hh = b_hh

    def zero_grad(self):
        h, d = self.hidden_size, self.input_size
        self.dW_ih = np.zeros((h, d))
        self.dW_hh = np.zeros((h, h))
        self.db_ih = np.zeros(h)
        self.db_hh = np.zeros(h)

    def __call__(self, x, h_prev_t):
        return self.forward(x, h_prev_t)

    def forward(self, x, h_prev_t):
        h_next =  h_prev_t @ self.W_hh.T + self.b_ih + x @ self.W_ih.T + self.b_hh
        out = self.activation.forward(h_next)
        return out

    def backward(self, dh_next, h_next, h_prev_l, h_prev_t):
        dz = self.activation.backward(dh_next, h_next)
        batch_size = dh_next.shape[0]
        # 1) Compute the averaged gradients of the weights and biases
        self.dW_ih += dz.T @ h_prev_l / batch_size
        self.dW_hh += dz.T @ h_prev_t / batch_size
        self.db_ih += np.sum(dz.T, axis=1) / batch_size
        self.db_hh += np.sum(dz.T, axis=1) / batch_size

        # 2) Compute dx, dh_prev_t
        dx = dz @ self.W_ih
        dh_prev_t = dz @ self.W_hh

        return dx, dh_prev_t
```

8. RNN Classifier (CMU 11-785)
    1. Example: many-to-one, multiple layers RNN Classifier
    2. RNN Classifier forward,
    3. RNN Classifier backward,
        1. An important principle for BPTT: very simple rule: for each node with multiple descendants in the computational graph, simply add up the delta vectors coming from all of the descendants.

4. 

The forward computation flow for the RNN

5. 

The backward computation flow for the RNN

6. Code: models/rnn_classifier.py

```
import numpy as np

from HW3.P1.mytorch.rnn_cell import *
from HW3.P1.mytorch.linear import *


class RNNPhonemeClassifier(object):
    """RNN Phoneme Classifier class."""

    def __init__(self, input_size, hidden_size, output_size, num_layers=2):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers

        # TODO: Understand then uncomment this code :)
        self.rnn = [
            (
                RNNCell(input_size, hidden_size)
                if i == 0
                else RNNCell(hidden_size, hidden_size)
            )
            for i in range(num_layers)
        ]
        self.output_layer = Linear(hidden_size, output_size)

        # store hidden states at each time step, [(seq_len+1) * (num_layers, batch_size, hidden_size)]
        self.hiddens = []

    def init_weights(self, rnn_weights, linear_weights):
```

```
        """Initialize weights.

        Parameters
        ----------
        rnn_weights:
                    [
                        [W_ih_l0, W_hh_l0, b_ih_l0, b_hh_l0],
                        [W_ih_l1, W_hh_l1, b_ih_l1, b_hh_l1],
                        ...
                    ]

        linear_weights:
                        [W, b]

        """
        for i, rnn_cell in enumerate(self.rnn):
            rnn_cell.init_weights(*rnn_weights[i])
        self.output_layer.W = linear_weights[0]
        self.output_layer.b = linear_weights[1].reshape(-1, 1)

    def __call__(self, x, h_0=None):
        return self.forward(x, h_0)

    def forward(self, x, h_0=None):
        """RNN forward, multiple layers, multiple time steps.
        x: input sequence: (N, T, D)
        self.x: cache (for backprop) of input sequence: (N, T, D)
        h0: initial hidden state (optional), zeros if not given: (L, N, H)
        self.hiddens: cache (for backprop) of computed hidden states (h_next): (T+1, L, N, H)
        hiddens: computed hidden states for all the layers (L, N, H)
        h_next: hidden state (N, H)
        logits: output (N, H_out)
        """
        N, T, D = x.shape
        L = self.num_layers
        H = self.hidden_size

        if h_0 is None:
            hiddens = np.zeros((L, N, H), dtype=np.float64)
        else:
            L, _, H = h_0.shape
            assert L == self.num_layers, H == self.hidden_size
            hiddens = h_0
        self.hiddens.append(hiddens.copy())
        self.x = x
        for t in range(T):
            for l in range(L):
                # Compute h_prev_l
                if l == 0:
                    h_prev_l = x[:, t, :]
                else:
                    h_prev_l = hiddens[l - 1, :, :]
                # Compute h_prev_t
                h_prev_t = hiddens[l]
                # Perform forward on RNN Cell
                h_next = self.rnn[l].forward(h_prev_l, h_prev_t)
                # Update hiddens for next time step
                hiddens[l] = h_next

            self.hiddens.append(hiddens.copy())

        logits = self.output_layer.forward(h_next)
        return logits

    def backward(self, delta):
        """RNN Back Propagation Through Time (BPTT).
        N: number of samples or batch size
        L: number of layers
        H: hidden dimension
        D: input dimension
        H_out: output dimension
        delta: Upstream gradient, dL/dy from linear layer before loss function
        i.e. in last RNN Cell, h_next -> y -> L: (N, H_out)
        self.hiddens: Contains h_next computed in each RNN Cell: (T+1, L, N, H)
        self.x: input sequence: (N, T, D)
        dh_next: contains upstream gradients: (L, N, H)
        for backprop, cache = (h_next, h_prev_l, h_prev_t, dh_next)
        """
        T = self.x.shape[1]
        L = self.num_layers
        H = self.hidden_size
        N, _ = delta.shape
        dh_next = np.zeros((L, N, H), dtype=np.float64)
        # Compute upstream gradient, dL/dh_next, for last RNN Cell
        dh_next[-1] = self.output_layer.backward(delta)
        # self.output_layer.backward(delta)
        for t in reversed(range(T)):
            for l in reversed(range(L)):
                # Compute variables in cache need for backprop
                h_next = self.hiddens[t + 1][l, :, :]
```

```
                h_prev_t = self.hiddens[t][l, :, :]
                # h_prev_l can come from either input sequence or h_next from previous cell, l-1
                if l == 0:
                    h_prev_l = self.x[:, t, :]
                else:
                    h_prev_l = self.hiddens[t + 1][l - 1, :, :]
                # Done computing variables in cache needed for backprop
                # Perform backprop on RNN Cell
                h_prev_l, h_prev_t = self.rnn[l].backward(
                    dh_next[l], h_next, h_prev_l, h_prev_t
                )
                # Update dh_next for the next previous time step
                dh_next[l] = h_prev_t
                # Add downstream gradient to the upstream gradient of the next previous cell step
                if l != 0:
                    dh_next[l - 1] += h_prev_l
        return dh_next / N


if __name__ == "__main__":
    input_size = 3
    hidden_size = 4
    output_size = 5
    rnn = RNNPhonemeClassifier(input_size, hidden_size, output_size)

    N = 10
    seq_len = 3
    x = np.random.random((N, seq_len, input_size))
    logits = rnn.forward(x)
    # print(logits.shape)
    # print(logits)
    delta = np.random.random((N, output_size))
    dh = rnn.backward(delta)
    print(dh)
    print(dh.shape)
```

	shape
W_{xh}	(D, H)
W_{hh}	(H, H)
x_t	(N, D)
h	(N, H)
b_h	(H,)
z_t	(N, H)

3. RNN Cell (CS231n)
    1. forward
        1. Given $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$
        2. Let $U = x_t \cdot W_{xh}$
        3. Let $V = h_{t-1} \cdot W_{hh}$
        4. $z_t = U + V + b_h$
        5. $h_t = \tanh\left(z_t\right)$
        6. Code: from cs231n/rnn_layers.py

```
def rnn_step_forward(x, prev_h, Wx, Wh, b):
    """
    Run the forward pass for a single timestep of a vanilla RNN that uses a tanh
    activation function.

    The input data has dimension D, the hidden state has dimension H, and we use
    a minibatch size of N.

    Inputs:
    - x: Input data for this timestep, of shape (N, D).
    - prev_h: Hidden state from previous timestep, of shape (N, H)
    - Wx: Weight matrix for input-to-hidden connections, of shape (D, H)
    - Wh: Weight matrix for hidden-to-hidden connections, of shape (H, H)
    - b: Biases of shape (H,)

    Returns a tuple of:
    - next_h: Next hidden state, of shape (N, H)
    - cache: Tuple of values needed for the backward pass.
    """
    next_h, cache = None, None
    ##############################################################################
    # TODO: Implement a single forward step for the vanilla RNN. Store the next  #
    # hidden state and any values you need for the backward pass in the next_h   #
    # and cache variables respectively.                                          #
    ##############################################################################
```

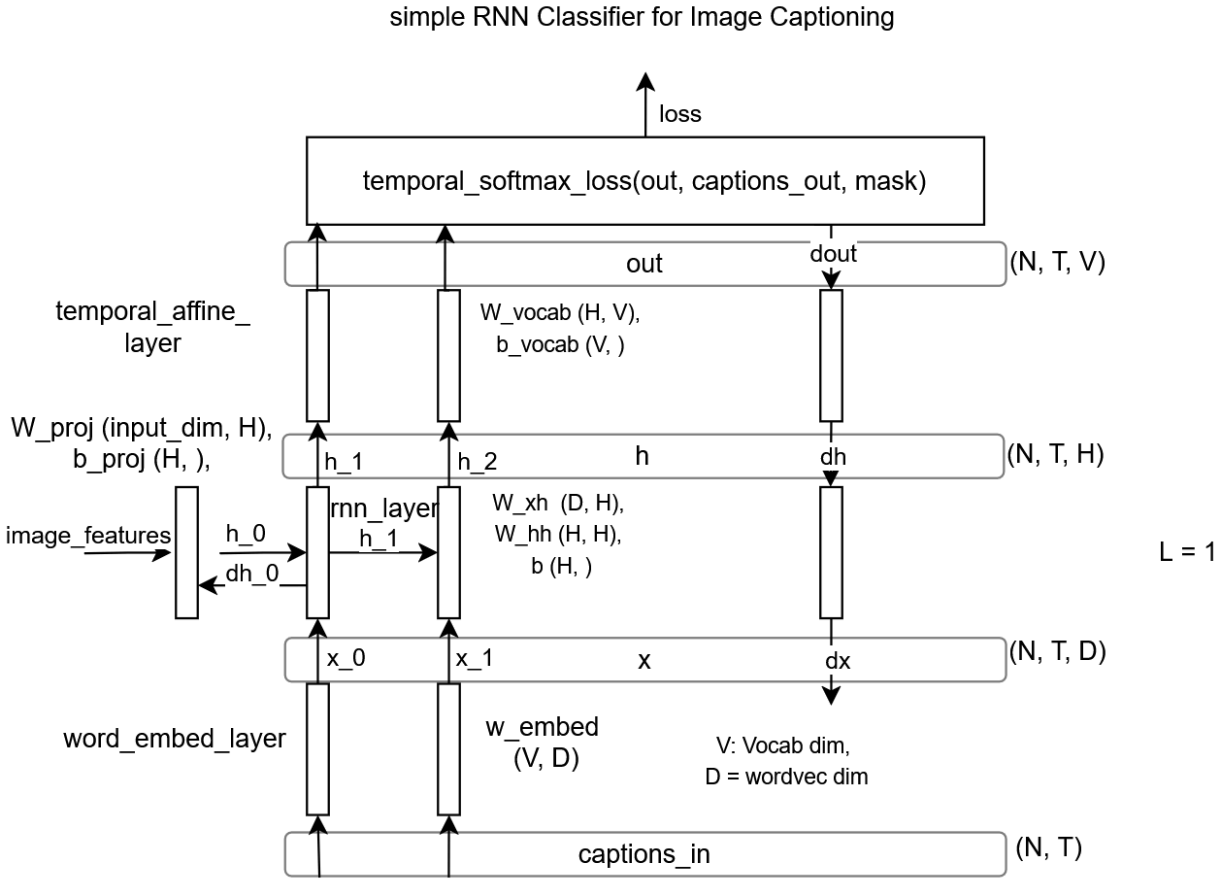
```
    z_t = x @ Wx + prev_h @ Wh + b
    next_h = np.tanh(z_t)
    # cache = (z_t, x, prev_h, Wx, Wh)
    cache = (next_h, x, prev_h, Wx, Wh)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return next_h, cache
```

2. backward
    1. $\frac{\partial L}{\partial z_t} = \frac{\partial L}{\partial h_t} \odot (1 - \tanh^2(z_t))$
    2. $\frac{\partial L}{\partial U} = \frac{\partial L}{\partial z_t}$
    3. $\frac{\partial L}{\partial V} = \frac{\partial L}{\partial z_t}$
    4. $\frac{\partial L}{\partial W_{xh}} = (\frac{\partial L}{\partial z_t}^T \cdot x_t)^T$
    5. $\frac{\partial L}{\partial W_{hh}} = (\frac{\partial L}{\partial z_t}^T \cdot h_{t-1})^T$
    6. $\frac{\partial L}{\partial x_t} = \frac{\partial L}{\partial z_t} W_{xh}^T$
    7. $\frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial z_t} W_{hh}^T$
    8. $\frac{\partial L}{\partial b} = (\frac{\partial L}{\partial z_t}).sum(axis = 0)$
    9. Code: from cs231n/rnn_layers.py

```
def rnn_step_backward(dnext_h, cache):
    """
    Backward pass for a single timestep of a vanilla RNN.

    Inputs:
    - dnext_h: Gradient of loss with respect to next hidden state
    - cache: Cache object from the forward pass

    Returns a tuple of:
    - dx: Gradients of input data, of shape (N, D)
    - dprev_h: Gradients of previous hidden state, of shape (N, H)
    - dWx: Gradients of input-to-hidden weights, of shape (D, H)
    - dWh: Gradients of hidden-to-hidden weights, of shape (H, H)
    - db: Gradients of bias vector, of shape (H,)
    """
    dx, dprev_h, dWx, dWh, db = None, None, None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for a single step of a vanilla RNN.      #
    #                                                                            #
    # HINT: For the tanh function, you can compute the local derivative in terms #
    # of the output value from tanh.                                             #
    ##############################################################################
    # z_t, x_t, prev_h, Wx, Wh = cache
    next_h, x_t, prev_h, Wx, Wh = cache
    dz = dnext_h * (1 - next_h**2)
    dx = dz @ Wx.T
    dprev_h = dz @ Wh.T
    dWx = x_t.T @ dz
    dWh = prev_h.T @ dz
    db = np.sum(dz, axis=0)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return dx, dprev_h, dWx, dWh, db
```

4. RNN Classifier (CS231n)

1. many-to-many architecture for image captioning:



2. RNN Classifier forward, code given below,
3. RNN Classifier backward (BPTT), code given below,
4. Code: from cs231n/rnn_layers.py

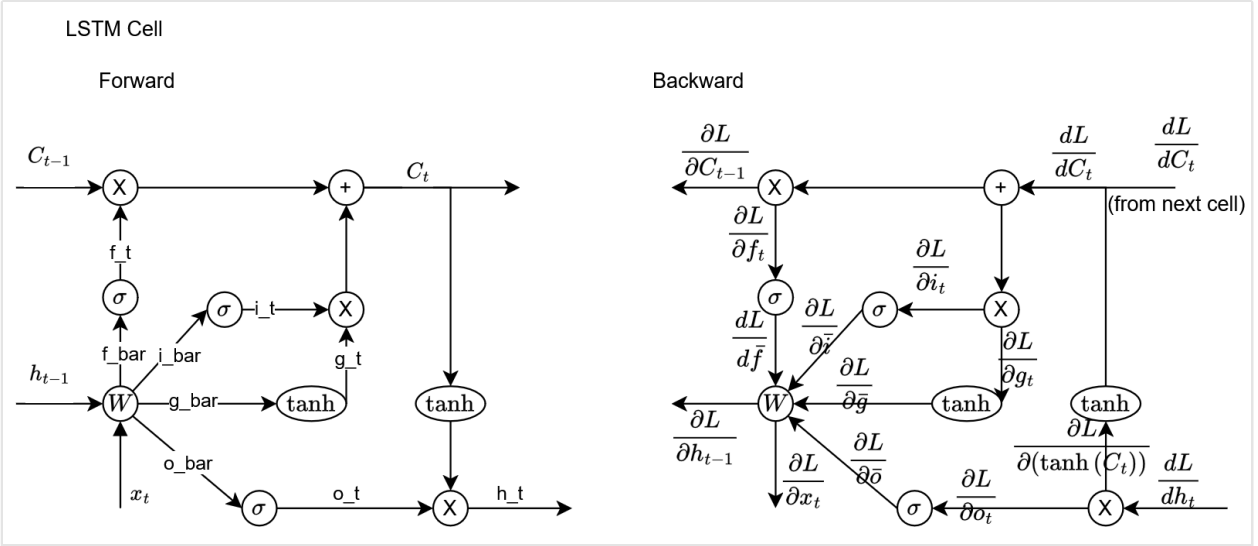
```
def rnn_forward(x, h0, Wx, Wh, b):
    """
    Run a vanilla RNN forward on an entire sequence of data. We assume an input
    sequence composed of T vectors, each of dimension D. The RNN uses a hidden
    size of H, and we work over a minibatch containing N sequences. After running
    the RNN forward, we return the hidden states for all timesteps.

    Inputs:
    - x: Input data for the entire timeseries, of shape (N, T, D).
    - h0: Initial hidden state, of shape (N, H)
    - Wx: Weight matrix for input-to-hidden connections, of shape (D, H)
    - Wh: Weight matrix for hidden-to-hidden connections, of shape (H, H)
    - b: Biases of shape (H,)

    Returns a tuple of:
    - h: Hidden states for the entire timeseries, of shape (N, T, H).
    - cache: Values needed in the backward pass
    """
    h, cache = [], []
    ##############################################################################
    # TODO: Implement forward pass for a vanilla RNN running on a sequence of    #
    # input data. You should use the rnn_step_forward function that you defined  #
    # above. You can use a for loop to help compute the forward pass.            #
    ##############################################################################
    prev_h = h0
    _, T, _ = x.shape
    for t in range(T):
        next_h, next_cache = rnn_step_forward(
            x[:, t, :], prev_h,
            Wx, Wh, b
        )
        h.append(next_h)
        cache.append(next_cache)
        prev_h = next_h
    h = np.array(h)
    h = h.transpose(1, 0, 2)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return h, cache


def rnn_backward(dh, cache):
```

```
    """
    Compute the backward pass for a vanilla RNN over an entire sequence of data.

    Inputs:
    - dh: Upstream gradients of all hidden states, of shape (N, T, H)

    Returns a tuple of:
    - dx: Gradient of inputs, of shape (N, T, D)
    - dh0: Gradient of initial hidden state, of shape (N, H)
    - dWx: Gradient of input-to-hidden weights, of shape (D, H)
    - dWh: Gradient of hidden-to-hidden weights, of shape (H, H)
    - db: Gradient of biases, of shape (H,)
    """
    dx, dh0, dWx, dWh, db = None, None, None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for a vanilla RNN running an entire      #
    # sequence of data. You should use the rnn_step_backward function that you   #
    # defined above. You can use a for loop to help compute the backward pass.   #
    ##############################################################################
    N, T, H = dh.shape

    dx = []
    # Extract D from x.shape
    _, x, _, _, _ = cache[0]
    _, D = x.shape
    curr_dx = np.zeros((N, D))
    dh0 = np.zeros((N, H))
    dWx = np.zeros((D, H))
    dWh = np.zeros((H, H))
    db = np.zeros(H)
    for t in reversed(range(T)):
        # Compute upstream gradients as input to RNN cell that computes local derivatives,
        # and produces dx, dprev_h, dWh, dWx and db
        if t == T - 1:
            dnext_h = dh[:, t, :]
        else:
            dnext_h = dh[:, t, :] + dprev_h
        curr_dx, dprev_h, prev_dWx, prev_dWh, prev_db = rnn_step_backward(dnext_h, cache[t])
        dWx += prev_dWx
        dWh += prev_dWh
        db += prev_db
        dx.append(curr_dx)
    dh0 = dprev_h
    # Because the way its elements are appended, resort the order based on ascending index of time
    dx.reverse()
    dx = np.array(dx)
    dx = dx.transpose(1, 0, 2)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return dx, dh0, dWx, dWh, db
```

LSTM

	shape
W_{xh}	(D, H)
W_{hh}	(H, H)
h_{t-1}	(N, H)
x_t	(N, D)
W	(D+H, 4H)
X	(N, H+D)
b	(4H,)
z	(N, 4H)

1. LSTMs and GRUs can use sequences of lengths upto around 80-90 to 100 tokens[8], beyond that Transformers perform better.
2. LSTM Cell
    1. $\mathbf{i}_t = \sigma(\mathbf{W}_{ix}\mathbf{x}_t + \mathbf{W}_{ih}\mathbf{h}_{t-1})$,
    2. $\mathbf{f}_t = \sigma(\mathbf{W}_{fx}\mathbf{x}_t + \mathbf{W}_{fh}\mathbf{h}_{t-1})$,
    3. $\mathbf{o}_t = \sigma(\mathbf{W}_{ox}\mathbf{x}_t + \mathbf{W}_{oh}\mathbf{h}_{t-1})$,
    4. $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_{cx}\mathbf{x}_t + \mathbf{W}_{ch}\mathbf{h}_{t-1})$,
    5. $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$,



LSTM computational graph

6. Forward
   1. $i_t = \sigma([x_t, h_{t-1}] \begin{bmatrix} W_{xi} \\ W_{hi} \end{bmatrix})$, input gate
   2. $f_t = \sigma([x_t, h_{t-1}] \begin{bmatrix} W_{xf} \\ W_{hf} \end{bmatrix})$, forget gate
   3. $o_t = \sigma([x_t, h_{t-1}] \begin{bmatrix} W_{xo} \\ W_{ho} \end{bmatrix})$, output gate
   4. $g_t = \tanh([x_t, h_{t-1}] \begin{bmatrix} W_{xg} \\ W_{hg} \end{bmatrix})$, 'gate' gate
   5. $\begin{pmatrix} \bar{i} \\ \bar{f} \\ \bar{o} \\ \bar{g} \end{pmatrix} = [x_t \quad h_{t-1}] \begin{bmatrix} W_{xi} & W_{xf} & W_{xo} & W_{xg} \\ W_{hi} & W_{hf} & W_{ho} & W_{hg} \end{bmatrix} + b$, shape: (n, 4h), where

      $z = \begin{pmatrix} \bar{i} \\ \bar{f} \\ \bar{o} \\ \bar{g} \end{pmatrix}, \quad X = [x_t \quad h_{t-1}], \quad W = \begin{bmatrix} W_{xi} & W_{xf} & W_{xo} & W_{xg} \\ W_{hi} & W_{hf} & W_{ho} & W_{hg} \end{bmatrix}$
   6. $i_t = \sigma(\bar{i})$
   7. $f_t = \sigma(\bar{f})$
   8. $o_t = \sigma(\bar{o})$
   9. $g_t = \tanh(\bar{g})$
   10. $C_t = f_t \odot C_{t-1} + i_t \odot g_t$
   11. $h_t = o_t \odot \tanh(C_t)$
   12. Code: from cs231n/rnn_layers.py

```python
def lstm_step_forward(x, prev_h, prev_c, Wx, Wh, b):
    """
    Forward pass for a single timestep of an LSTM.

    The input data has dimension D, the hidden state has dimension H, and we use
    a minibatch size of N.

    Inputs:
    - x: Input data, of shape (N, D)
    - prev_h: Previous hidden state, of shape (N, H)
    - prev_c: previous cell state, of shape (N, H)
    - Wx: Input-to-hidden weights, of shape (D, 4H)
    - Wh: Hidden-to-hidden weights, of shape (H, 4H)
    - b: Biases, of shape (4H,)

    Returns a tuple of:
    - next_h: Next hidden state, of shape (N, H)
    - next_c: Next cell state, of shape (N, H)
    - cache: Tuple of values needed for backward pass.
    """
    next_h, next_c, cache = None, None, None
    #############################################################################
    # TODO: Implement the forward pass for a single timestep of an LSTM.        #
    # You may want to use the numerically stable sigmoid implementation above.  #
    #############################################################################
    x_stack = np.hstack((x, prev_h))
    w_stack = np.vstack((Wx, Wh))
```

```
    z = x_stack @ w_stack + b  # shape: (N, 4*H)
    i_bar, f_bar, o_bar, g_bar = np.split(z, 4, axis=1)
    i_t = sigmoid(i_bar); f_t = sigmoid(f_bar); o_t = sigmoid(o_bar); g_t = np.tanh(g_bar)
    next_c = f_t * prev_c + i_t * g_t
    next_h = o_t * np.tanh(next_c)
    cache = (prev_h, x, prev_c, next_c,
             i_bar, f_bar, o_bar, g_bar,
             i_t, f_t, o_t, g_t,
             Wx, Wh)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return next_h, next_c, cache
```

7. Backward
    1. cache (to store in forward): $C_t, \bar{g}, \bar{o}, \bar{f}, \bar{i}, x_t, h_{t-1}, C_{t-1}, g_t, o_t, f_t, i_t, W_x, W_h$.
    2. $A \cdot B$ denotes matrix multiplication, $A \odot B$ denotes element-wise multiplication, $\sigma(x) = \text{sigmoid(x)} = \frac{1}{1+e^{-x}}$.
    3. $\frac{dL}{do_t} = \frac{\partial L}{\partial h_t}\frac{\partial h_t}{\partial o_t} = \frac{\partial L}{\partial h_t} \odot \tanh(C_t)$
    4. $\frac{\partial L}{\partial(\tanh C_t)} = \frac{\partial L}{\partial h_t}\frac{\partial h_t}{\partial(\tanh C_t)} = \frac{\partial L}{\partial h_t} \odot o_t$
    5. $\frac{\partial L}{\partial C_t} += \frac{\partial L}{\partial(\tanh(C_t))}\frac{\partial(\tanh(C_t))}{\partial C_t} = \frac{\partial L}{\partial(\tanh C_t)} \odot (1 - \tanh^2 C_t)$
    6. $\frac{\partial L}{\partial f_t} = \frac{\partial L}{\partial C_t} \odot C_{t-1}$
    7. $\frac{\partial L}{\partial C_{t-1}} = \frac{\partial L}{\partial C_t} \odot f_t$
    8. $\frac{\partial L}{\partial i_t} = \frac{\partial L}{\partial C_t} \odot g_t$
    9. $\frac{\partial L}{\partial g_t} = \frac{\partial L}{\partial C_t} \odot i_t$
    10. $\frac{\partial L}{\partial \bar{g}} = \frac{\partial L}{\partial g_t} \odot (1 - \tanh^2 \bar{g})$
    11. $\frac{\partial L}{\partial \bar{o}} = \frac{\partial L}{\partial o_t}\frac{\partial \sigma(\bar{o})}{\partial \bar{o}} = \frac{\partial L}{\partial o_t} \odot \sigma(\bar{o}) \odot (1 - \sigma(\bar{o}))$
    12. $\frac{\partial L}{\partial \bar{f}} = \frac{\partial L}{\partial f_t}\frac{\partial \sigma(\bar{f})}{\partial \bar{f}} = \frac{\partial L}{\partial f_t} \odot \sigma(\bar{f}) \odot (1 - \sigma(\bar{f}))$
    13. $\frac{\partial L}{\partial \bar{i}} = \frac{\partial L}{\partial i_t}\frac{\partial \sigma(\bar{i})}{\partial \bar{i}} = \frac{\partial L}{\partial i_t} \odot \sigma(\bar{i}) \odot (1 - \sigma(\bar{i}))$
    14. $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial z} \cdot W^T$
    15. $\frac{\partial L}{\partial W} = (\frac{\partial L}{\partial z}^T \cdot X)^T$
    16. $\frac{\partial L}{\partial b} = (\frac{\partial L}{\partial z}).sum(axis = 0)$
    17. Code: from cs231n/rnn_layers.py

```
def lstm_step_backward(dnext_h, dnext_c, cache):
    """
    Backward pass for a single timestep of an LSTM.

    Inputs:
    - dnext_h: Gradients of next hidden state, of shape (N, H)
    - dnext_c: Gradients of next cell state, of shape (N, H)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient of input data, of shape (N, D)
    - dprev_h: Gradient of previous hidden state, of shape (N, H)
    - dprev_c: Gradient of previous cell state, of shape (N, H)
    - dWx: Gradient of input-to-hidden weights, of shape (D, 4H)
    - dWh: Gradient of hidden-to-hidden weights, of shape (H, 4H)
    - db: Gradient of biases, of shape (4H,)
    """
    dx, dh, dc, dWx, dWh, db = None, None, None, None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for a single timestep of an LSTM.       #
    #                                                                            #
    # HINT: For sigmoid and tanh you can compute local derivatives in terms of  #
    # the output value from the nonlinearity.                                    #
    ##############################################################################
    prev_h, x, prev_c, next_c, i_bar, f_bar, o_bar, g_bar, i_t, f_t, o_t, g_t, Wx, Wh = cache
    do_t = dnext_h * np.tanh(next_c)
    dnext_c += dnext_h * o_t * (1 - np.tanh(next_c)**2)
    df_t = dnext_c * prev_c
    dprev_c = dnext_c * f_t
    di_t = dnext_c * g_t
    dg_t = dnext_c * i_t
    dg_bar = dg_t * (1 - np.tanh(g_bar)**2)
    do_bar = do_t * (1 - sigmoid(o_bar)) * sigmoid(o_bar)
    df_bar = df_t * (1 - sigmoid(f_bar)) * sigmoid(f_bar)
    di_bar = di_t * (1 - sigmoid(i_bar)) * sigmoid(i_bar)
    dz = np.hstack((di_bar, df_bar, do_bar, dg_bar))
    W = np.vstack((Wx, Wh))
    dx_ = dz @ W.T
    _, D = x.shape
    # _, H = prev_h.shape
    dx = dx_[:, :D]
    dprev_h = dx_[:, D:]
    X = np.hstack((x, prev_h))
```

```
    dW = (dz.T @ X).T
    dWx = dW[:D, :]
    dWh = dW[D:, :]
    db = dz.sum(axis=0)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return dx, dprev_h, dprev_c, dWx, dWh, db
```

3. LSTM Classifier
    1. Forward
        1. Code: from cs231n/rnn_layers.py

```
def lstm_forward(x, h0, Wx, Wh, b):
    """
    Forward pass for an LSTM over an entire sequence of data. We assume an input
    sequence composed of T vectors, each of dimension D. The LSTM uses a hidden
    size of H, and we work over a minibatch containing N sequences. After running
    the LSTM forward, we return the hidden states for all timesteps.

    Note that the initial cell state is passed as input, but the initial cell
    state is set to zero. Also note that the cell state is not returned; it is
    an internal variable to the LSTM and is not accessed from outside.

    Inputs:
    - x: Input data of shape (N, T, D)
    - h0: Initial hidden state of shape (N, H)
    - Wx: Weights for input-to-hidden connections, of shape (D, 4H)
    - Wh: Weights for hidden-to-hidden connections, of shape (H, 4H)
    - b: Biases of shape (4H,)

    Returns a tuple of:
    - h: Hidden states for all timesteps of all sequences, of shape (N, T, H)
    - cache: Values needed for the backward pass.
    """
    h, cache = [], []
    ##############################################################################
    # TODO: Implement the forward pass for an LSTM over an entire timeseries.    #
    # You should use the lstm_step_forward function that you just defined.       #
    ##############################################################################
    _, T, _ = x.shape
    prev_h = h0
    prev_c = np.zeros_like(prev_h)
    for t in range(T):
        next_h, next_c, next_cache = lstm_step_forward(x[:, t, :], prev_h, prev_c, Wx, Wh, b)
        h.append(next_h)
        cache.append(next_cache)
        prev_h = next_h
        prev_c = next_c
    h = np.array(h)
    h = h.transpose(1, 0, 2)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return h, cache
```

    2. Backward
        1. Code: from cs231n/rnn_layers.py

```
def lstm_backward(dh, cache):
    """
    Backward pass for an LSTM over an entire sequence of data.]

    Inputs:
    - dh: Upstream gradients of hidden states, of shape (N, T, H)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient of input data of shape (N, T, D)
    - dh0: Gradient of initial hidden state of shape (N, H)
    - dWx: Gradient of input-to-hidden weight matrix of shape (D, 4H)
    - dWh: Gradient of hidden-to-hidden weight matrix of shape (H, 4H)
    - db: Gradient of biases, of shape (4H,)
    """
    dx, dh0, dWx, dWh, db = None, None, None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for an LSTM over an entire timeseries.   #
    # You should use the lstm_step_backward function that you just defined.      #
    ##############################################################################
    dx = []
    N, T, H = dh.shape
    dnext_c = np.zeros((N, H))
    x = cache[0][1]
    _, D = x.shape
    dh0 = np.zeros((N, H))
    dWx = np.zeros((D, 4 * H))
    dWh = np.zeros((H, 4 * H))
```

```
    db = np.zeros((4 * H, ))
    for t in reversed(range(T)):
        if t == T - 1:
            dnext_h = dh[:, t, :]
        else:
            dnext_h = dh[:, t, :] + dprev_h
        dprev_x, dprev_h, dprev_c, dprev_Wx, dprev_Wh, dprev_b = lstm_step_backward(dnext_h, dnext_c,
cache[t])
        dnext_c = dprev_c
        dWx += dprev_Wx
        dWh += dprev_Wh
        db += dprev_b
        dx.append(dprev_x)
    dh0 = dprev_h
    dx.reverse()
    dx = np.array(dx)
    dx = dx.transpose(1, 0, 2)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return dx, dh0, dWx, dWh, db
```

Transformers

1. see Transformers

Generative models

1. "What I cannot create I do not understand" - Richard Feynman. Generative modeling is based on the contrapositive of this "what I understand I should be able to create".
2. Given observed samples x from a distribution of interest, the goal of generative model is to learn to model its true distribution $p(x)$.
3. Generative modelling
    1. Z is some standard random variable.
    2. f_θ is a neural network with parameters θ.
    3. X is a parametric RV whose distribution depends on $f_\theta(Z)$.
    4. How is the training? **Training any probability model is always the same, its just maximum likelihood estimation**. We want to pick the neural network parameters so as to maximize the log likelihood of the dataset. We'll assume that our dataset consists of N independent samples x1 to x_N.
    5. Training: MLE: we want θ to maximize log likelihood of dataset.

        1. $$\begin{aligned}\log Pr(x_1, \ldots, x_n; \theta) &= \sum_i \log Pr_x(x_i; \theta) \\ &= \sum_i \log \int_z Pr_x(x_i | Z = z; \theta) Pr_z(z) \mathrm{dz}\end{aligned}$$

        2.

Autoencoder

1. How NOT to train an autoencoder
    1. Define a reconstruction loss, call it $L(\tilde{x}, x)$.
    2. Train enc. and dec. to minimize loss on the training set.
    3. Evaluate loss on the holdout dataset.
        1. This won't work because, the trained autoencoder can reconstruct the dataset but it does not extract useful features for the input, so there is no guarantee that semi-supervised learning would work, and random generation won't work either we're just spitting random variable z the our thought experiment autoencoder will only produce noise not an interesting output.
        2. So we would need to think much harder on how to train an autoencoder.
        3. The right way to train an autoencoder is discovered by two independent researchers in 2014. The answer is probabilistic modelling, specifically generative modelling.
2.

Diffusion models

1. Connection to VAEs
    1. Diffusion models can be considered as special case of hierarchical VAEs.

    2. However, in diffusion models:
        1. The encoder is fixed.
        2. The latent variables have the same dimension as the data.
        3. The denoising model is shared across different timestep.
        4. The model is trained with some reweighting of the variational bound.
2. DDPM is an image-generation model
    1. DDPM is inspired by indirect sampling technique like **Gibbs sampling**, to sample a uniform distribution and feed it into a Markov Chain.
    2. Generating images.
    3. Sampling images from a simple prior.
    4. Modeling the distrib of the data X of interest.
    5. Computing the prob of data p(x), x is an image.
    6. In contrast to discriminative models that models p(y|x).
    7. Training: Diffusion to get the input.
        1. Sample an image x from the training set.
        2. Sample noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$.
        3. Sample a variance $\beta \in (0, 1)$.
        4. Get the input $x_t = \sqrt{1-\beta}x + \sqrt{\beta}\epsilon$.
        5. reparametrization trick: $x = mean + std\_dev * Z$ to sample from normal distribution. Z is drawn from a standard normal distribution.
        6. Note: reparametrization trick is a useful mental step to do backprop through a sampling process. While the gradient of an individual sample indeed does not exist or isn't meaningful, it does exist in expectation, and the expectation is all you need to backprop. Papers that refer this as a trick may come from the years 2012-2014.
    8. Training: Denoising to get the output
        1. Feed the input into the network.
        2. Network: U-Net f_θ.
        3. Predict the noise $f_\theta(x_t, t)$.
        4. Loss is MSE.
        5. input to model: $x_t = \sqrt{1-\beta}x + \sqrt{\beta}\epsilon$
        6. output: $\tilde{\epsilon}$, $\tilde{x} = \frac{x_t - \sqrt{\beta}\tilde{\epsilon}}{\sqrt{1-\beta}}$, denoising: noisy image x_t minus the noise $\sqrt{\beta}\epsilon$.
    9. Inference: Iterative denoising
    10. DDPM does not learn the variance, just the mean of the noise ϵ.
    11. ELBO continued
        1. We need the explicit form of $q(\mathbf{x}_t, \mathbf{x}_0)$ and $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ for sampling.
        2. We present the result here: $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$, or equivalently $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_0$, where $\bar{\alpha}_t = \prod_{i=1}^{t}\alpha_i$ and $\epsilon_0 \sim \mathcal{N}(0, \mathbf{I})$.
        3. $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}|\mu_q(\mathbf{x}_t, \mathbf{x}_0), \mathbf{\Sigma}_q(t))$, where $\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_t}\mathbf{x}_0$, and $\mathbf{\Sigma}_q(t) = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{I} = \sigma_q^2(t)\mathbf{I}$.
        4. First way of modeling
            1. From the ELBO, we know that we have to compute the KL divergence term. p_θ is what we can set for training, and we know that $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is Gaussian, so for convenience we formulate p_θ as a Gaussian, with the same variance as $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and a learnable mean $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}|\mu_\theta(\mathbf{x}_t, t), \sigma_q^2(t)\mathbf{I})$ where $\mu_\theta(\mathbf{x}_t, t)$ is a neural network parametrized by θ.
            2. Now we have $D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \frac{1}{2\sigma_q^2(t)}||\mu_q(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2$.
            3. To match the form of mean in $\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_t}\mathbf{x}_0$ we set $\mu_\theta(\mathbf{x}_t) = \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_t}\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)$, where $\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)$ is another neural network (still parametrized by θ) that predicts the clean image $\mathbf{x}_0$ from the noisy image $\mathbf{x}_t$ and time t.
        5. Second way of modeling
            1. Note that we set our neural network ($\hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)$) for predicting the image. We can actually learn to predict the noise. To see that, from $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon_0$ we can obtain $\mathbf{x}_0 =$ - and put it in $\mu_q(\mathbf{x}_t, \mathbf{x}_0)$, and then get
            2. $\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}\sqrt{\alpha_t}}\epsilon_0$.
            3. So similarly we can design our mean estimator μ_θ as
            4. $\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}\sqrt{\alpha_t}}\hat{\epsilon}_\theta(\mathbf{x}_t, t)$.

References:

1. Unsupervised Representation Learning by Predicting Image Rotations. Gidaris, et. al. Ecole des Ponts ParisTech.
2. Understanding Diffusion Models: A Unified Perspective, Calvin Luo.
3. CS182 Deep Learning, UC Berkeley, Sergey Levine, 2021.
4. CS330 Variational Inference and Generative models, Chelsea Finn, Stanford, 2022.
5. https://jiegroup-genai.readthedocs-hosted.com/en/latest/diffusion/index.html#denoising-diffusion-probabilisitc-model-ddpm, DDPM, Umass Amherst.
6. Object classification uses Sigmoid+binary_cross_entropy, why not keep same as DETR using Softmax+cross_entropy? https://github.com/fundamentalvision/Deformable-DETR/issues/72#issuecomment-886408142
7. DDPM in Keras, https://keras.io/examples/generative/ddpm/
8. Lecture 19, Transformers and GNNs, CMU 11-785 F22.
9. The Principles of Diffusion Models, Chieh-Hsin Lai et al.